Rank & Select

Advanced Data Structures – Summer 2026 – Lecture 2

Ragnar {Groot Koerkamp}, Stefan Walzer, Stefan Hermann

2026-04-27 Mon 14:00

Last week: Models of computation

- Big-O notation
- Word RAM model
  - Applications & limitations

Today

- Bit vectors
  - storing bits
- Rank
  - counting 1-bits
- Select
  - finding 1-bits
- Engineering Rank
  - making it fast
- Elias-Fano coding
  - storing integers efficiently

Bit vectors

Bit vectors

Definition 1.

A static *bit vector* is an *implicit* data structure storing n bits $b_0, \ldots, b_{n-1}$ with constant overhead.

- $\texttt{get}(i)$: returns i'th bit b_i

Implemented using e.g. `Vec<u64>`.

```
fn get(data: &Vec<u64>, i: usize) -> bool {
    let word = data[i / 64];          // compiled to >> 6
    let idx = i % 64;                 // compiled to & 0b0011_1111
    return ((word >> idx) & 1) > 0;
}
```

Little Endian: the *low*-order bits are first in memory.

Binary Rank & Select

Binary Rank & Select

Definition 2.

A *rank* query on a bit vector counts 1-bits *before* a position i (for $0 \leq i \leq n$).

- $\mathsf{rank}(i) := \sum_{0 \leq j < i} b_j$

Definition 3.

A *select* query on a bit vector returns the position of the j'th 1 (zero-based, for $0 \leq j < \mathsf{rank}(n)$).

- $\mathsf{select}(j) :=$ index i such that $b_i = 1$ and $\mathsf{rank}(i) = j$.

Also: rank_0 and select_0 for counting/finding 0-bits.

Question 1.

What is $\mathsf{rank}_1(\mathsf{select}_1(j))$?

And $\mathsf{select}_1(\mathsf{rank}_1(i))$?

Binary Rank & Select

Answer 1.

$\mathsf{rank}_1(\mathsf{select}_1(j))$ equals j.

$\mathsf{select}_1(\mathsf{rank}_1(i))$ finds the position of the first 1 at position $\geq i$.

Binary Rank & Select: Goal

Theorem 1: ***Goal of this lecture*.**

There exists a data structure that answers rank and select queries:

- in constant $O(1)$ time,
- using succinct $o(n)$ bits of additional space.

Rank: first steps

Example 1: *Compute everything*.

Compute the answer on-demand:

- No space overhead
- $O(n/\log n)$ query time
  - We pop-count $w = \Theta(\log n)$ bits at a time.

Example 2: *Store everything*.

Pre-compute and store all answers:

- n *words* overhead
- $O(1)$ query time

Rank: Binary search

Example 3.

Store positions of all 1-bits. Then binary search to get the number of them at position $< i$:

- k words overhead, for $k := \mathsf{rank}(n)$.
- $O(\log k)$ query time

Rank: Checkpoints

Example 4.

Store a *checkpoint* every s bits: $R_k := \mathsf{rank}(k \cdot s)$.

- n/s *words* overhead
- $O(s/\log n)$ query time:

$$\mathsf{rank}(i) = R_{\lfloor \frac{i}{s} \rfloor} + \sum_{s\lfloor \frac{i}{s} \rfloor \leq j < i} b_j$$

For $s = w = \Theta(\log n)$:

Rank: Checkpoints

Example 4.

Store a *checkpoint* every s bits: $R_k := \mathsf{rank}(k \cdot s)$.

- n/s *words* overhead
- $O(s/\log n)$ query time:

$$\mathsf{rank}(i) = R_{\lfloor \frac{i}{s} \rfloor} + \sum_{s\lfloor \frac{i}{s} \rfloor \leq j < i} b_j$$

For $s = w = \Theta(\log n)$:

- n/w words, or n bits overhead.
  - *Compact*, but not *succinct*
- $O(1)$ queries

Rank: Checkpoints

Example 4.

Store a *checkpoint* every s bits: $R_k := \mathsf{rank}(k \cdot s)$.

- n/s *words* overhead
- $O(s/\log n)$ query time:

$$\mathsf{rank}(i) = R_{\lfloor \frac{i}{s} \rfloor} + \sum_{s\lfloor \frac{i}{s} \rfloor \leq j < i} b_j$$

For $s = w = \Theta(\log n)$:

- n/w words, or n bits overhead.
  - *Compact*, but not *succinct*
- $O(1)$ queries

- Adjacent checkpoints R_k are similar!

Rank: A succinct version (Jacobson 1988)

Definition 4: *Succinct rank*.

- Blocks of size $s = w$.
- Super blocks of size $s' = w^2$.

Rank: A succinct version (Jacobson 1988)

Definition 4: *Succinct rank*.

- Blocks of size $s = w$.
- Super blocks of size $s' = w^2$.
- Store w-bit super block ranks $R'_k := \mathsf{rank}(k \cdot s')$.
  - $\lceil n/s' \rceil \cdot w = O(n/\log n) = o(n)$ bits

Rank: A succinct version (Jacobson 1988)

Definition 4: *Succinct rank*.

- Blocks of size $s = w$.
- Super blocks of size $s' = w^2$.
- Store w-bit super block ranks $R'_k := \mathsf{rank}(k \cdot s')$.
  - $\lceil n/s' \rceil \cdot w = O(n/\log n) = o(n)$ bits
- Store $\log_2 s'$-bit block ranks relative to super block:

$$R_k = \mathsf{rank}(k \cdot s) - R'_{\lfloor k/w \rfloor} \leq s'.$$

  - $\lceil n/s \rceil \cdot \log_2(s') = O(n/w \cdot \log_2 w) = o(n)$ bits

Rank: A succinct version (Jacobson 1988)

Definition 4: *Succinct rank*.

- Blocks of size $s = w$.
- Super blocks of size $s' = w^2$.
- Store w-bit super block ranks $R'_k := \mathsf{rank}(k \cdot s')$.
  - $\lceil n/s' \rceil \cdot w = O(n/\log n) = o(n)$ bits
- Store $\log_2 s'$-bit block ranks relative to super block:

$$R_k = \mathsf{rank}(k \cdot s) - R'_{\lfloor k/w \rfloor} \leq s'.$$

  - $\lceil n/s \rceil \cdot \log_2(s') = O(n/w \cdot \log_2 w) = o(n)$ bits
- Constant time queries:

$$\mathsf{rank}(i) = R'_{\lfloor i/s' \rfloor} + R_{\lfloor i/s \rfloor} + \sum_{s\lfloor \frac{i}{s} \rfloor \leq j < i} b_j$$

Rank: Emulating popcount

- Originally, constant time w-bit popcount was not assumed.

Rank: Emulating popcount

- Originally, constant time w-bit popcount was not assumed.
- Instead, we can take $s = (\log_2 n)/2$, and pre-compute popcounts for all 2^s masks:
  - $2^s = \sqrt{n}$ patterns
  - s offsets per pattern
  - $\log_2 s$ bits per count

$$2^s \cdot s \cdot \log_2 s = O(\sqrt{n} \cdot \log_2 n \cdot \log_2 \log_2 n) = o(n).$$

Engineering Rank: minimizing space usage

In practice, we don't need *succinct*. *Compact* with small constant overhead is enough. See QuadRank slides.

Engineering Rank: minimizing space usage

In practice, we don't need *succinct*. *Compact* with small constant overhead is enough. See QuadRank slides.

- $s = 512$: Store a 64-bit checkpoint every 512-bits of data:
  - $64/512 = 12.5\%$ space overhead;
  - pop-counting 512 bits is sufficiently fast.

Engineering Rank: minimizing space usage

In practice, we don't need *succinct*. *Compact* with small constant overhead is enough. See QuadRank slides.

- $s = 512$: Store a 64-bit checkpoint every 512-bits of data:
  - $64/512 = 12.5\%$ space overhead;
  - pop-counting 512 bits is sufficiently fast.
- Smaller, $s' = 2^{16}$: 16-bit checkpoints, and $s' = 2^{16}$-bit super blocks storing 64-bit checkpoints. (Kurpicz 2022)
  - $16/512 + 64/2^{16} = 3.2\%$ overhead.

Engineering Rank: minimizing space usage

In practice, we don't need *succinct*. *Compact* with small constant overhead is enough. See QuadRank slides.

- $s = 512$: Store a 64-bit checkpoint every 512-bits of data:
  - $64/512 = 12.5\%$ space overhead;
  - pop-counting 512 bits is sufficiently fast.
- Smaller, $s' = 2^{16}$: 16-bit checkpoints, and $s' = 2^{16}$-bit super blocks storing 64-bit checkpoints. (Kurpicz 2022)
  - $16/512 + 64/2^{16} = 3.2\%$ overhead.
- Faster count: checkpoints to the *middle* of each block: (Gottlieb and Reinert 2025)
  - popcount only 256 bits.

Engineering Rank: minimizing space usage

In practice, we don't need *succinct*. *Compact* with small constant overhead is enough. See QuadRank slides.

- $s = 512$: Store a 64-bit checkpoint every 512-bits of data:
  - $64/512 = 12.5\%$ space overhead;
  - pop-counting 512 bits is sufficiently fast.
- Smaller, $s' = 2^{16}$: 16-bit checkpoints, and $s' = 2^{16}$-bit super blocks storing 64-bit checkpoints. (Kurpicz 2022)
  - $16/512 + 64/2^{16} = 3.2\%$ overhead.
- Faster count: checkpoints to the *middle* of each block: (Gottlieb and Reinert 2025)
  - popcount only 256 bits.
- Fewer cache misses: store checkpoints *inside* each block: (Laws et al. 2024)
  - 1 instead of 2 cache misses
  - $16/(512 - 16) + 64/2^{16} = 3.3\%$ overhead.

Engineering Rank: minimizing space usage

In practice, we don't need *succinct*. *Compact* with small constant overhead is enough. See QuadRank slides.

- $s = 512$: Store a 64-bit checkpoint every 512-bits of data:
  - $64/512 = 12.5\%$ space overhead;
  - pop-counting 512 bits is sufficiently fast.
- Smaller, $s' = 2^{16}$: 16-bit checkpoints, and $s' = 2^{16}$-bit super blocks storing 64-bit checkpoints. (Kurpicz 2022)
  - $16/512 + 64/2^{16} = 3.2\%$ overhead.
- Faster count: checkpoints to the *middle* of each block: (Gottlieb and Reinert 2025)
  - popcount only 256 bits.
- Fewer cache misses: store checkpoints *inside* each block: (Laws et al. 2024)
  - 1 instead of 2 cache misses
  - $16/(512 - 16) + 64/2^{16} = 3.3\%$ overhead.
- Simple & fast classic: Rank 9 (Vigna 2008)
  - 64-bit blocks with 9-bit checkpoints; 7 of them make a `u64`.

Select

Select

Assume we have k 1-bits.

- Compute all answers on-demand: $O(n/\log n)$ queries.
- Store all answers: $O(k)$ words.
  - Sufficient if $k = o(n/\log n)$.

Definition 5: *Select in word*.

The position of i'th 1-bit in a word can be found in constant time (in the "modern" word RAM model) using

```
(1 << i).deposit_bits(word).trailing_zeros()
```

which uses `pdep` and `tzcnt` instructions (wikipedia, deposit_bits).

Succinct Select (Clark and Munro 1996)

Definition 6: *Clark's select*.

- Variable-sized super-blocks B_i containing $b := \log^2 n$ 1-bits

$$\mathsf{select}(i) = \left(\sum_{0 \le j < \lfloor i/b \rfloor} |B_j| \right) + \mathsf{select}(B_{\lfloor i/b \rfloor}, i - b\lfloor i/b \rfloor).$$

  - Prefix sums take $n/b \cdot w = O(n/\log n) = o(n)$ bits.
- *Sparse* blocks with $|B| \ge \log^4 n$:
  - Store answers directly in $bw = \Theta(\log^3 n)$ bits.
  - $\log^4$ bits in block, so $o(n)$ overall.
- *Dense* blocks with $|B| < \log^4 n$: *next slide*

Succinct Select (Clark and Munro 1996)

Definition 7: *(Clark's select ctd.) handling sparse blocks*.

Dense blocks with $|B| < \log^4 n$:

Split again into variable-sized blocks of $b' = \sqrt{\log n}$ ones.

- Prefix sums take $\log \log^4 n$ bits per element, $O(k/b' \cdot \log \log^4 n) = o(n)$ total
- If sparse, size $\geq \frac{1}{2} \log n$:
  - Store $b' \log_2(\log^4 n)$-bit numbers, $o(\log n)$ total
- If dense, size $< \frac{1}{2} \log n$:
  - Use `select_in_word`
  - Clark's select stores a lookup table like for popcount.

Succinct Select (Clark and Munro 1996)

Definition 8: ***(Clark's select ctd.) queries*.**

To answer $\mathsf{select}(j)$:

- Find the super block $\lfloor j/b \rfloor$.
- If sparse: read answer.
- If dense: find block $\lfloor j/b' \rfloor$.
  - Read offset of superblock.
  - Read offset of block inside superblock.
  - If sparse block: read the position relative to the block.
  - If dense block: popcount inside the block.

Theorem 2.

There exists a data structure with $o(n)$ space overhead that allows $O(1)$ rank and select queries.

Select in practice: Darray

- Super blocks containing $b = 2^{10}$ ones.
- Sparse if length $\geq 2^{16}$
  - Use $2^6 = 64$ bits to encode each position; $64 \cdot 2^{10} = 2^{16}$ bits

Select in practice: Darray

- Super blocks containing $b = 2^{10}$ ones.
- Sparse if length $\geq 2^{16}$
  - Use $2^6 = 64$ bits to encode each position; $64 \cdot 2^{10} = 2^{16}$ bits
- Dense blocks (len $< 2^{16}$):
  - Store position of every $2^5 = 32$'nd 1, relative to start (10 bits each).
  - Linear scan ($< 2^{16}$) from there to find the exact answer.
  - The last sparse/dense split from Clark's select (previous slide) is skipped.

Further reading

- QuadRank slides and paper (Groot Koerkamp 2026)
- Ben Langmead's teaching material on bitvectors, rank, and select.
  - With YouTube videos.

Next: Elias-Fano coding

Bibliography

Clark, David R, and J Ian Munro. 1996. “Efficient Suffix Trees on Secondary Storage.” In *Proceedings of the Seventh Annual Acm-Siam Symposium on Discrete Algorithms*, 383–91.

Gottlieb, Simon Gene, and Knut Reinert. 2025. “Engineering Rank Queries on Bit Vectors and Strings.” *Algorithms for Molecular Biology* 20 (1). https://doi.org/10.1186/s13015-025-00291-9.

Groot Koerkamp, Ragnar. 2026. “Quadrank: Engineering a High Throughput Rank.” arXiv. https://doi.org/10.48550/ARXIV.2602.04103.

Jacobson, Guy Joseph. 1988. “Succinct Static Data Structures.” Carnegie Mellon University.

Kurpicz, Florian. 2022. “Engineering Compact Data Structures for Rank and Select Queries on Bit Vectors.” In *SPIRE 2022*, 257–72. Springer International Publishing. https://doi.org/10.1007/978-3-031-20643-6_19.

Laws, Matthew D., Jocelyn Bliven, Kit Conklin, Elyes Laalai, Samuel McCauley, and Zach S. Sturdevant. 2024. “Spider: Improved Succinct Rank and Select Performance.” In *SEA 2024*, 301:21:1–21:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. https://doi.org/10.4230/LIPICS.SEA.2024.21.

Vigna, Sebastiano. 2008. “Broadword Implementation of Rank/Select Queries.” In *WEA 2008*, edited by Catherine C. McGeoch, 154–68. Berlin, Heidelberg: Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-68552-4_12.